



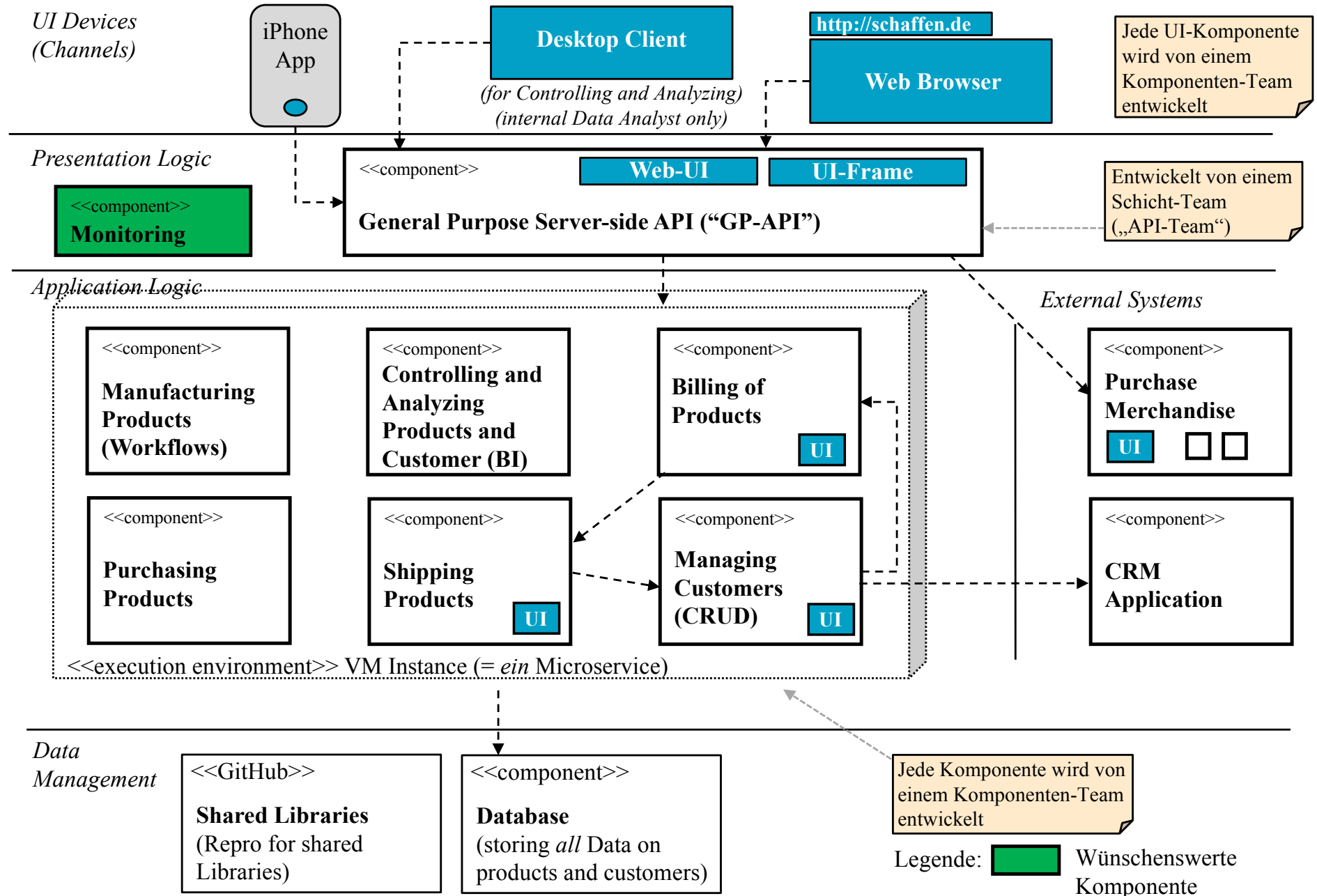
Case Study Microservices – Analyse der Architektur der Firma *WirSchiffenDas*

GmbH & Co KGaA

– internes Papier (v3.0) des Entwicklungsteams

**Vertrauensvoll aufbereitet und bereitgestellt von:
Prof. Dr. Sascha Alda (c/o 2022 - 2026)**

Baustein-Sicht der Software-Architektur der Firma WirSchiffenDas GmbH & Co KGaA





Beschreibung der Produkte und Kunden-Akteure

- Entity „Produkt“:
 - Die Firma stellt Elemente (=Produkte) für Yachten her und verkauft diese für verschiedene Kundensegmente (siehe unten).
 - Verschiedene Produkte werden hier zusammengefasst:
 - Motoren für Yachten sowie
 - Einrichtungselemente für Yachten (beide *nur* über das Web-Interface bestellbar);
 - Kataloge über Yachten sowie
 - kleine Modelle von Yachten (beide *nur* über die Mobile App bestellbar)
- Akteure „Kunden“:
 - Großkunden (kaufen vordefinierte Motoren und Einrichtungselemente über eine konventionelle Web-Anwendung, die in einem Web Browser abläuft)
 - Kleinkunden (benutzen ausschließlich eine Mobile App zum Erwerb von Katalogen und Modellen)
 - Premiumkunden (agieren wie Großkunden, können zusätzlich die Herstellung von Elementen *interaktiv* über eine Web-Anwendung mitgestalten)



Weitere Infos zu dem zentralen Microservice

- Der zentrale Microservice kapselt die Geschäftssicht (*business layer*), hier hat man sich an einem technischen Schnitt orientiert.
- Für diesen und auch für zukünftige Microservices hat man ein zentrales GitHub-Repository für „Shared Libraries“ bereitgestellt, was die Unabhängigkeit von Microservices vereinfachen soll“. Das GitHub-Repository ist der (einzige) Grundbaustein für die moderne CI/CD-Infrastruktur der Abteilung, bei der nur noch das finale Deployment manuell durchgeführt werden muss. Quality-Checks gibt es nicht.
- Bei dem zentralen Microservice ist man stolz, auf eine *moderne* Container-Technology aufzubauen, was die Skalierung und auch das Deployment insgesamt verbessern soll. „Je nach Bedarf werden die internen Komponenten ja unabhängig mit-deployed“, so die Auffassung des Software-Entwicklers.
- GP-API benutzt die internen Komponenten des Microservice, man greift auf IP-basierte REST-Endpunkte direkt zu (z.B. „100.2.33.255/products“; eine Versionierung gibt es nicht). Gelegentlich verwendet man auch XML-RPC oder MQ-basierte Endpunkte. Durch die Verwendung von REST sei die Verwendung einer Registry nicht notwendig. Auch zukünftige Microservices interagieren untereinander über solche Endpunkte.



Generelle Aspekte

- Diese Komponente wird von den Ingenieuren des Unternehmens zur Durchführung der Konstruktion (engl. Manufacturing) der Produkte für Groß- und Premiumkunden verwendet.
- In dieser Komponente gibt es für *jedes* Produkt (Motoren und Einrichtungselemente) für Yachten einen intern Workflow (Ablauf) zur Konstruktion der jeweiligen Produkte. Dieser interne Ablauf wird durch eine **Konstruktions-Komponente** realisiert. Eine Konstruktion eines Produkts kann innerhalb der Komponente *verwaltet* werden.
- Bei Motoren werden in dem Ablauf vor allem das „Optional Equipment“ abgefragt. Siehe dazu auch die Informationen aus dem Datenblatt für den Motor „Diesel Engine 2000 M96“.
- Diese Komponente hat nur eine API, keine UI. Für eine Interaktion mit dem Benutzer im Web-Browser stellt GPI-API ein eigenes Web-Frontend („Web-UI“) bereit.
- Für Premiumkunden gibt es eine **Chat-Komponente**, um Motoren interaktiv zu gestalten. Somit können Ingenieure und Premiumkunden online und gleichzeitig wichtige Entwurfsentscheidungen für das „Optional Equipment“ diskutieren.



Die Analyse-Komponente

- Es gibt ferner eine interne **Analyse-Komponente**, welche die aktuelle Spezifikation eines Produkts verifiziert. Die Laufzeit einer Analyse aufgrund interner Simulationsverfahren ist recht hoch, so dass es hier häufig zu Verzögerungen kommt, so dass die gesamte Komponente zur Durchführung der Konstruktion eines Produkts aufgrund der internen *engen* Kopplung häufig still steht, also *nicht* mehr *responsive* ist, bis die Analysekomponente sich wieder „erholt“ hat und eine Antwort liefert. Dies betrifft vor allem die Analyse von Motoren. Über einen Timeout-Mechanismus hat man nachgedacht, diesen aber generell in der Architektur als nicht relevant angesehen. Mögliche Gründen für einen Stillstand kann man später in einem lokalen Log-File ablesen.
- Da die Analyse-Komponente von allen Workflows verwendet wird, kommt es gerade im Hochbetrieb der Anwendung (Oktober bis Dezember eines Jahres) zu sehr unangenehmen Verzögerungen und vereinzelt sogar zu Ausfällen im Ablauf. Eine *individuelle Skalierung* dieser Analysekomponente ist erwünscht durch die Ingenieure, wurde aber bisher durch die Bereichsleitung des Bereichs „Technik“ abgelehnt, da man die Skalierung der *gesamten* Anwendung vorziehen möchte. Eine individuelle Skalierung sei aus technischer Sicht schlichtweg *nicht* möglich.
- Auch die **Ausfallsicherheit** kann man nicht verbessern (O-Ton eines Insiders, der mit dem Bereichsleiter sprach: „*Damit sollen wir leben und halt nicht zu viel analysieren!*“). Dennoch ist man im Entwicklerteam der Komponente offen für die Anwendung von Pattern und Software-Bibliotheken zur Verbesserung der Ausfallsicherheit.

Beschreibung der Komponente „Manufacturing Products“

Die „Microservice“-Vision



- Die in „Manufacturing Products“ angedeuteten **Komponenten entsprechen Java-Packages**; eine Komponenten-Orientierung hat man nicht realisiert bisher. Das Entwicklerteam wäre aber gerne an der Übernahme der Komponenten-Orientierung interessiert, gerne auch an einer zukünftigen Entwicklung der Komponenten als *eigenständige Microservices*.
- Konstruktionsspezifische Daten eines Produkts möchte man gerne „näher“ zu der Komponente „Manufacturing Products“ abspeichern, um somit nicht zu abhängig von der globalen Datenbank auf dem Layer „Data Management“ zu sein. Gegenüber einem universalen Datenmodell steht man sehr *kritisch* gegenüber.
- Man ist sich bewusst, dass durch das neue Daten-Management man nun auch Abhängigkeiten zu Komponenten bzw. zu den Entwicklerteams der Komponenten schafft, da fast alle auf (zumindest Teile der) konstruktionsspezifische Daten zugreifen. Das Team gibt sich offen und kundenorientiert! Man möchte hier gerne als **„Zulieferer“ von Daten** fungieren, sollten abhängige Teams *außerhalb* der Produktentwicklung hier Wünsche z.B. bei der Darstellung oder Aufbereitung von Daten haben. Innerhalb der Produktentwicklung will man partnerschaftlich arbeiten.
- Der Manager des Entwicklerteams gibt noch an, dass man mit dem „Controlling“-Team ein eher „angespanntes“ Verhältnis hat. O-Ton: „Eine „Zulieferung“ kommt hier nicht in Frage! Die sollen mal schön brav unsere Datenmodelle verwenden! Eine Abhängigkeit besteht zur (Teil-)Komponente „Managing Products“, da werden oft Daten von uns bezogen“. Hier gibt er sich sorglos: „Die müssen sich nicht anpassen oder was zuliefern, hier realisieren auf unserer Seite einen Adapter!“



Order Fullfillment (Auftragsabwicklung)

- Die Komponenten „Shipping Products“ und „Billing of Products“ sind interne Komponenten zum Versand bzw. zur Rechnungsstellung von Produkten. Diese werden durch Vertriebsarbeiter verwendet. Beide Komponenten bieten eine UI sowie eine REST-API an. Die Komponente „Shipping Products“ hat hohe Skalierungsanforderungen, da die Komponente sehr lange Bearbeitungszeiten insbesondere in der Sommerzeit hat.
- Die Komponente „Managing Customers (CRUD)“ ist eine interne Komponente um Kunden intern zu verwalten. Sie wird von Vertriebsmitarbeitern, der Produktentwicklung und der Geschäftsführung verwendet. Diese Komponente verwendet die globale Datenbank auf dem Layer „Data Management“. Auch diese Komponente bietet eine UI und eine REST-API an.
- All diese Komponenten bieten Services an, die den Workflow „Order Fullfillment“ unterstützen. Nach Aussage der Bereichsleitung, handelt es sich hier um einen *manuellen* und recht fehleranfälligen Prozess, da die einzelnen Schritte manuell über die recht heterogenen User-Interfaces der Komponenten abgewickelt werden.
- Eine Integration der einzelnen User Interfaces erfolgt serverseitig in der GP-API-Komponente, diese werden in einem **UI-Frame** integriert, die dann in Web-Browsern gerendert werden kann.
- Die Geschäftsführung spricht sich für eine *moderne* Orchestration der Services an mit einem einheitlichen Frontend, gerne mit „modernsten KI-Support“ (O-Ton!)



- Möglicher Use Case: „**Kunde bestellt Produkt**“ (mit selektierten Endpunkten)

Managing
Customers

- 1. Kunde wird geprüft (ggf. neu angelegt)

```
POST /customers
GET /customers/{id}
POST /customers/{id}/validate
```

- 2. Bestellung wird erstellt

Billing of
Products

- 3. Rechnung wird erzeugt

```
public invoiceID create_Invoice(
    Order order )
```

- 4. Zahlungsstatus überprüfen

Shipping of
Products

- 5. Versand wird vorbereitet

```
create_Shipment( order : Order )
track_Shipment ( shipmentID : Int)
```

- 6. Trackingnummer wird generiert



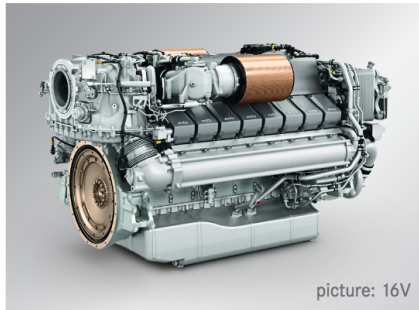
Beschreibung der weiteren Komponenten

- Die **Komponente „Purchasing Products“** ist eine Komponente für Kunden zum Kauf von Produkten, die im weiteren Verlauf den Workflow „Order Fullfillment“ ansteuert. Diese Komponente hat nur eine API, keine UI. Für eine Interaktion mit dem Benutzer im Web-Browser stellt GPI-API ein eigenes Web-Frontend („Web-UI“) bereit.
- Kleinkunden können über die Komponente „Purchasing Products“ auch über ein iPhone Modelle und Kataloge bestellen. Von der IT-Seite wünscht man sich eine Aufteilung der Komponente (z.B. um eine weitere Komponente „Purchasing Models and Catalogs“) zur Reduktion der Entwicklungsarbeiten.
- Die Geschäftsführung verwendet *auch* die **Komponente „Controlling and Analyzing Products and Customer (BI)“** über eine separate Desktop-Anwendung.
- Die Komponente „Purchase Merchandise“ ist eine vom zentralen Microservice entkoppelte, „experimentelle“ Software (man nutzt immer neueste und unterschiedliche OS-Software). Sie kann ebenfalls von Kunden über eine bereitgestellte UI verwendet werden.
 - Hier verwendet man einen hochskalierten Ansatz: Jede Business-Methode wurde in einen einzelnen Microservices abgebildet, die über das Netzwerk kommunizieren.
 - Die Komponente umfasst 167 Business-Methoden.
 - Die Interaktion mit dem Benutzer erfolgt direkt mit der Web-basierten UI des Microservice, die über einen Endpunkt aus der GP-API abgerufen werden kann und in einem Web-Browser gerendert wird.

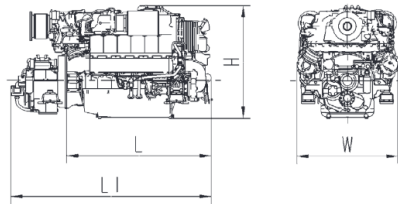
Datenblatt für den Yacht-Motor „Diesel Engine 2000 M96“



Diesel Engines 10V/12V/16V 2000 M96



picture: 16V



Dimensions and Masses

2000 M96

10V

12V ¹⁾

12V ²⁾

16V

2000 M96 - with Gearbox type ³⁾

10V - ZF 2050

12V - ZF 2060 ¹⁾

12V - ZF 2060 ²⁾

16V - ZF 3060

Optional Equipment

Starting system

Auxiliary PTO

Oil system

Fuel System

Cooling System

Exhaust System

Mounting System

Engine Management System

Monitoring/Control System

Power Transmission

Gearbox Options

Air starter*

Alternator, 140A or 190A, 28V, 2 pole, bilgepump, on-engine PTOs

Oil replenishment system, diverter valve for duplex filter

Duplex fuel pre-filter, diverter valve for fuel filter, monitoring fuel leakage

Coolant preheating system freestanding or engine mounted, integr. seawater gearbox piping

90° Exhaust bellows discharge rotatable

Resilient mounts at driving end

In compliance with Classification Society Regulations

BlueVision | NewGeneration

Torsionally resilient coupling

Reverse reduction gearbox, el. actuated, gearbox mounts, trolling mode for dead-slow propulsion, free auxiliary PTO, hydraulic pump drives

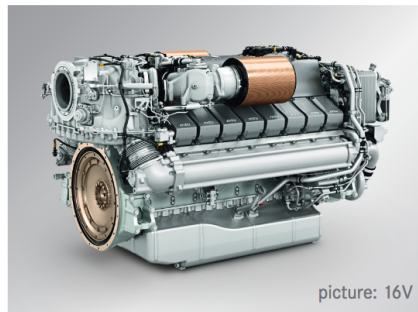
*only applies to 12V and 16V cylinder configurations

Quelle: Adaptiert von <https://www.mtu-online.com/mtu/anwendungen/yacht-diesel-motoren/index.de.html>

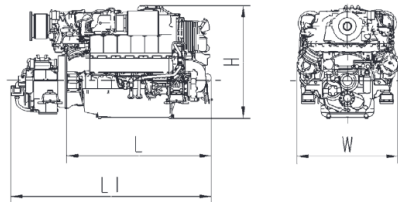


Bezug Analyse-Komponente zum Optional Equipment

Diesel Engines 10V/12V/16V 2000 M96



picture: 16V



Dimensions and Masses

2000 M96

10V

12V ¹⁾

12V ²⁾

16V

2000 M96 - with Gearbox type ³⁾

10V - ZF 2050

12V - ZF 2060 ¹⁾

12V - ZF 2060 ²⁾

16V - ZF 3060

Optional Equipment

Starting system

Auxiliary PTO

Oil system

Fuel System

Cooling System

Exhaust System

Mounting System

Engine Management System

Monitoring/Control System

Power Transmission

Gearbox Options

Air starter*

Alternator, 140A or 190A, 28V, 2 pole, bilgepump, on-engine PTOs

Oil replenishment system, diverter valve for duplex filter

Duplex fuel pre-filter, diverter valve for fuel filter, monitoring fuel leakage

Coolant preheating system freestanding or engine mounted, integr. seawater gearbox piping

90° Exhaust bellows discharge rotatable

Resilient mounts at driving end

In compliance with Classification Society Regulations

BlueVision | NewGeneration

Torsionally resilient coupling

Reverse reduction gearbox, el. actuated, gearbox mounts, trolling mode for dead-slow propulsion, free auxiliary PTO, hydraulic pump drives

*only applies to 12V and 16V cylinder configurations

O-Ton des zuständigen Ingenieurs, der im Rahmen einer ersten Anforderungsanalyse interviewt wurde:

„Die Analyse-Komponente für Komponenten ist deswegen so komplex, weil für jedes optionale Equipment ein eigenständiger und abgeschlossener Algorithmus zur Validierung und Simulation der Konfiguration durchgeführt werden muss. Aktuell sind diese Algorithmen in *einer* Klasse integriert. Ob man das nicht mal auftrennen könnte, **indem man die Algorithmen in einer Choreographie nebenläufig abarbeitet**? Die übrigen Komponenten sollte also *responsive* bleiben! Möchte auch gerne die aktuellen Status der einzelnen Algorithmen sehen können, das ist in der aktuellen Implementierung nicht möglich. Auch ein Retry eines einzelnen Algorithmus sollte möglich sein. Auch für eine Überwachung (Monitoring) der Services spreche ich mich aus.“

Quelle: Adaptiert von <https://www.mtu-online.com/mtu/anwendungen/yacht-diesel-motoren/index.de.html>

Beschreibung der Komponente „Manufacturing Products“

Monitoring



- Das **Monitoring** der gesamten Anwendung hält das Entwicklerteam für sinnvoll, gerade dann, wenn es zu einer Aufteilung nach Microservices kommt! Auch hier ist die Bereichsleitung sehr zögerlich. Man betrachtet eine Eigenentwicklung für das Monitoring als sinnvoll an (oder gibt es Produkte dazu?). Außer bei der Analyse-Komponente hat man kein Logging bisher realisiert.